
kubernetes - Task 1

Ehsan Borhani Nia

در این پروژه من سه سرویس Rancher، Traefik و GitLab Runner را روی یک کلاستر Kubernetes سه‌گره‌ای نصب و راه‌اندازی کردم. هدف من این بود که سرویس‌ها در Namespace‌های جداگانه اجرا شوند، Podهای اصلی در وضعیت Running قرار بگیرند، Rancher از طریق HTTPS قابل دسترسی باشد و GitLab Runner در پنل GitLab به صورت Online نمایش داده شود.

در این پروژه از ubuntu 3 استفاده شد:

- 1. ubuntu server02 → worker
- 2. ubuntu server03 → worker
- 3. helm و ubuntu server04 → Control Plane → kubectl

Manifest

بیشتر Manifestهای Kubernetes چهار بخش اصلی دارند:

apiVersion

kind: نوع Resource را مشخص می‌کند؛ Namespace، ServiceAccount، Deployment، Service، IngressClass یا Secret.

metadata: مهم‌ترین فیلدهای آن عبارت‌اند از:

- name: نام Resource
- namespace: محل ایجاد Resource
- labels: برچسب‌هایی برای دسته‌بندی و اتصال Resource‌ها
- annotations: تنظیمات تکمیلی مورد استفاده Controllerها

spec: وضعیت Resource را تعریف می‌کند. برای مثال، تعداد Replicaها، نام Image، Portها، Selectorها، محدودیت منابع و نوع Service داخل این بخش نوشته می‌شوند.

من ارتباط بین Deployment و Service را با Label و Selector برقرار کردم. Deployment روی Pod ها Label قرار می‌دهد و Service با Selector همان Pod ها را پیدا کرده و ترافیک را به آن‌ها ارسال می‌کند.

Traefik های Manifest

Traefik در این پروژه نقش Ingress Controller را دارد. یعنی درخواست HTTPS وارد کلاستر می‌شود، Traefik بر اساس Host و قوانین Ingress آن را دریافت می‌کند و سپس درخواست را به Service مربوط به Rancher می‌فرستد.

namespace.yaml فایل

در این فایل Namespace اختصاصی Traefik را ایجاد کردم:

```
apiVersion: v1
kind: Namespace
metadata:
name: traefik
```

استفاده از Namespace جدا باعث شد Resource های Traefik از Rancher و GitLab Runner جدا و مدیریت آن‌ها آسون تر بشود.

rbac.yaml فایل

Traefik برای مشاهده Ingress ها، Service ها، Secret های TLS و EndpointSlice ها به دسترسی Kubernetes API نیاز دارد. در این بخش Resource های زیر را تعریف کردم:

- ServiceAccount برای هویت Pod های Traefik
- ClusterRole برای تعیین دسترسی‌های موردنیاز
- ClusterRoleBinding برای اتصال ClusterRole به ServiceAccount

من فقط مجوزهایی را در نظر گرفتم که Traefik برای خواندن و مانیتور کردن Resource های شبکه نیاز دارد. Traefik با این دسترسی‌ها می‌تواند Ingress مربوط به Rancher و Endpoint آماده آن را شناسایی کند.

فایل ingressclass.yaml

در این فایل یک IngressClass با نام traefik تعریف کردیم. Controller این کلاس به شکل زیر است:

traefik.io/ingress-controller

وجود این Resource مشخص می‌کند که Ingress هایی با traefik ingressClassName: باید توسط Traefik پردازش شوند.

فایل deployment.yaml

در Pod، Deployment، اصلی Traefik را تعریف کردیم. مهم‌ترین تنظیمات آن عبارت‌اند از:

- تعداد Replica برابر با 1
- Image برابر با traefik:v3.7.11
- فعال‌سازی Kubernetes Ingress Provider
- EntryPoint عادی روی Port داخلی 8000
- EntryPoint امن روی Port داخلی 8443
- Port مدیریتی و Health Check روی 9000
- فعال‌سازی Liveness Probe و Readiness Probe
- تعیین CPU و Memory Request/Limit

Readiness Probe باعث می‌شود Service فقط زمانی ترافیک را به Pod ارسال کند که Traefik واقعاً آماده پاسخ‌گویی باشد. Liveness Probe نیز در صورت از کار افتادن برنامه به Kubernetes اجازه می‌دهد Container را دوباره راه‌اندازی کند.

Selector مربوط به Deployment و Label مربوط به Pod هر دو app: traefik هستند. بنابراین Deployment می‌تواند Replica های خودش را مدیریت کند و Service نیز همان Pod ها را انتخاب می‌کند.

فایل‌ها و تنظیمات Rancher

من Rancher را با Helm Chart نسخه 2.15.0 و App Version برابر با v2.15.0 نصب کردم. برای جلوگیری از مشکل دانلود، Chart را به صورت فایل محلی rancher-2.15.0.tgz نگه داشتم.

فایل rancher-values.yml

این فایل یک Kubernetes Manifest کامل نیست؛ بلکه فایل Values است که تنظیمات ورودی Helm Chart را مشخص می‌کند. Helm این مقادیر را با Template های Chart ترکیب کرده و Resource های نهایی Kubernetes را تولید می‌کند.

من تعداد Replica را به دلیل محدودیت منابع آزمایشگاه برابر با 1 قرار دادم. مقدار hostname آدرس Rancher را مشخص می‌کند و sslip.io نام دامنه را به IP سرور Control Plane تبدیل می‌کند.

برای سازگاری کامل Ingress با Traefik، تنظیمات زیر نیز باید در بخش Annotation های Ingress ذخیره شوند:

```
kubernetes.io/ingress.class: traefik
```

```
traefik.ingress.kubernetes.io/router.entrypoints: websecure
```

```
traefik.ingress.kubernetes.io/router.tls: "true"
```

Annotation اول مشخص می‌کند که Ingress توسط Traefik مدیریت شود. Annotation دوم Router را به EntryPoint امن متصل می‌کند و Annotation سوم TLS را برای Router فعال می‌کند.

Secret های TLS

من برای HTTPS از Secret استفاده کردم. دو Secret اصلی عبارت بودند از:

- `tls-rancher-ingress`: شامل Private Key و Certificate مربوط به Rancher
- `tls-ca`: شامل CA Certificate مورد اعتماد Rancher

Secret اول از نوع `kubernetes.io/tls` است و Ingress برای TLS Termination از آن استفاده می‌کند. به دلایل امنیتی، من Private Key، رمز Bootstrap و محتوای Secret ها را داخل گزارش یا فایل های عمومی پروژه قرار نمی‌دهم.

Resource های تولیدشده توسط Helm

Helm با استفاده از Chart و فایل Values چند Resource ایجاد کرد

بعد از نصب، Pod اصلی Rancher به وضعیت Running 1/1 رسید و پاسخ HTTPS به شکل زیر تأیید شد:

HTTP/2 200

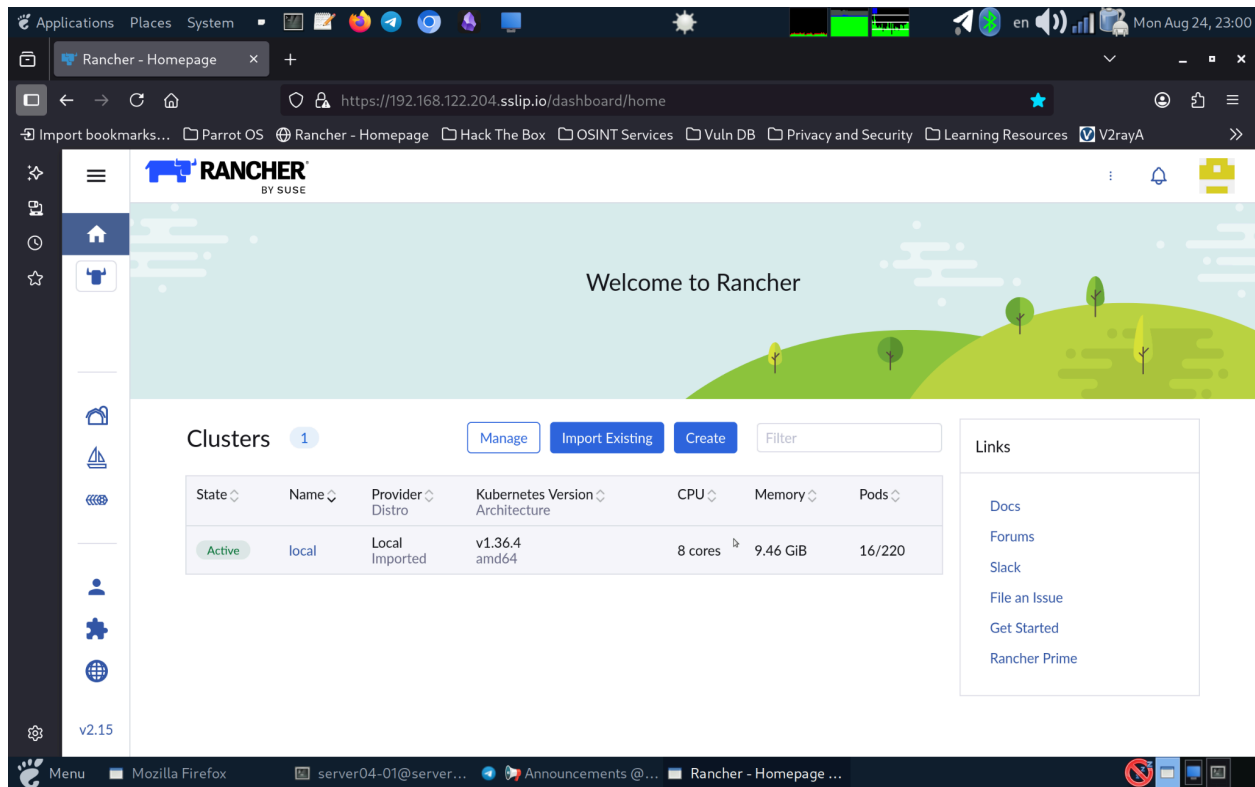
این نتیجه نشان داد که Rancher Service، TLS Secret، Ingress، Traefik و Rancher Pod به درستی به یکدیگر متصل شده‌اند.

فایل‌ها و تنظیمات GitLab Runner

من GitLab Runner را به صورت Pod داخل Kubernetes نصب کردم؛ بنابراین نیازی به نصب بسته gitlab-runner با apt روی server04 نبود. فرمان Helm روی server04 اجرا شد، اما Runner Pod روی Worker Node اجرا شد.

من از Helm Chart نسخه 0.92.0 و GitLab Runner نسخه 19.3.0 استفاده کردم. فایل محلی Chart برابر با gitlab-runner-0.92.0.tgz بود.

نتایج نهایی را به صورت اسکرین شات در پایین آورده شده است:



```
x-content-type-options: nosniff

server04-01@server04:~$ kubectl get pods -n cattle-system -o wide
NAME                                READY   STATUS              RESTARTS   AGE   IP              NODE           NOMINATED
NODE   READINESS GATES
helm-operation-2fts5                 0/2     Init:0/1            0           6s    <none>          server02        <none>
helm-operation-76tmz                 0/2     Completed           0           14m   10.244.1.53     server02        <none>
helm-operation-889zc                 0/2     Completed           0           20m   10.244.1.38     server02        <none>
helm-operation-d27pc                 0/2     Completed           0           15m   10.244.0.6      server04        <none>
helm-operation-fbk5j                 0/2     Init:0/1            0           69s   <none>          server03        <none>
helm-operation-nmgx8                 0/2     Completed           0           17m   10.244.1.43     server02        <none>
helm-operation-tvg8k                 0/2     Completed           0           18m   10.244.1.41     server02        <none>
helm-operation-vdbkm                 0/2     Error               0           19m   10.244.2.41     server03        <none>
helm-operation-vwnq7                 0/2     Init:ContainerStatusUnknown 0           28m   10.244.1.34     server02        <none>
helm-operation-wl6ts                 0/2     Completed           0           16m   10.244.1.46     server02        <none>
helm-operation-x8lwr                 0/2     Completed           0           21m   10.244.1.36     server02        <none>
rancher-76fd95cdcb-2l6qv             0/1     Error               0           34m   10.244.1.23     server02        <none>
rancher-76fd95cdcb-6vv29             0/1     Error               0           36m   10.244.2.28     server03        <none>
```

```

<none>
rancher-76fd95cdcb-8stld 0/1 Error 0 27m 10.244.2.35 server03 <none>
<none>
rancher-76fd95cdcb-crmh2 0/1 Error 0 28m 10.244.1.35 server02 <none>
<none>
rancher-76fd95cdcb-twhbt 0/1 Error 0 33m 10.244.2.33 server03 <none>
<none>
rancher-76fd95cdcb-vf2gp 1/1 Running 0 8m30s 10.244.1.59 server02 <none>
<none>
rancher-webhook-6746d4c6b5-vnklk 1/1 Running 0 18m 10.244.1.42 server02 <none>
<none>
server04-01@server04:~$ kubectl get pods -n traefik -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
traefik-5dbcf58487-2kwsn 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-2t7mb 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-4w4wz 0/1 Evicted 0 8m47s <none> server03 <none> <none>
traefik-5dbcf58487-4w6s7 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-4xnpv 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-4xzn9 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-665x9 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-78vwr 0/1 Evicted 0 8m47s <none> server03 <none> <none>
traefik-5dbcf58487-88vdb 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-8vb55 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-92d9b 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-92jvg 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-9cnx2 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-9kl6r 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-9smq9 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-c79r6 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-c8sxs 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-cb2rc 0/1 Evicted 0 8m45s <none> server03 <none> <none>

```

```

traefik-5dbcf58487-fm8nn 0/1 Evicted 0 8m44s <none> server03 <none> <none>
traefik-5dbcf58487-grwx2 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-hm8bp 0/1 Evicted 0 8m47s <none> server03 <none> <none>
traefik-5dbcf58487-hqls8 0/1 Evicted 0 8m44s <none> server03 <none> <none>
traefik-5dbcf58487-ht9f7 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-jd5lc 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-jvpss 0/1 Evicted 0 8m47s <none> server03 <none> <none>
traefik-5dbcf58487-lkr65 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-m2jkm 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-pdqrm 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-q5r9d 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-qtx5b 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-rn842 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-rqnwz 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-vdk9w 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-5dbcf58487-wcrxv 0/1 Evicted 0 8m47s <none> server03 <none> <none>
traefik-5dbcf58487-wm6j6 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-wv248 0/1 Evicted 0 8m47s <none> server03 <none> <none>
traefik-5dbcf58487-wvtg9 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-x6wlm 0/1 Completed 0 9m58s 10.244.2.43 server03 <none> <none>
traefik-5dbcf58487-xqj7f 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-xw7w2 0/1 Evicted 0 8m46s <none> server03 <none> <none>
traefik-5dbcf58487-z29kv 0/1 Evicted 0 8m49s <none> server03 <none> <none>
traefik-5dbcf58487-z9dv9 0/1 Evicted 0 8m48s <none> server03 <none> <none>
traefik-5dbcf58487-zv8lm 0/1 Evicted 0 8m45s <none> server03 <none> <none>
traefik-684776cd57-ft6m7 1/1 Running 0 4m26s 10.244.2.47 server03 <none> <none>
server04-01@server04:~$ kubectl get ingress -n cattle-system
NAME CLASS HOSTS ADDRESS PORTS AGE
rancher traefik 192.168.122.204.sslip.io 80, 443 114m
server04-01@server04:~$

```

server04-01@server04: ~/kubernetes-week1/manifests/gitlab-runner

File Edit View Search Terminal Help

Every 5.0s: kubectl get pods -n gitlab-runner -o wide

server04: Mon Aug 24 20:30:53 2026

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS
gitlab-runner-76cbb5d5d-xk65c	1/1	Running	0	6m53s	10.244.1.70	server02	<none>	<none>

Webhooks

Access tokens

Repository

Merge requests

CI/CD

Package and

> Deploy freezes

Add a freeze period to prevent unintended releases during a period of time for a given environment. You must update the deployment jobs in .gitlab-ci.yml according to the deploy freezes added here. Learn more. Specify deploy freezes using cron syntax.

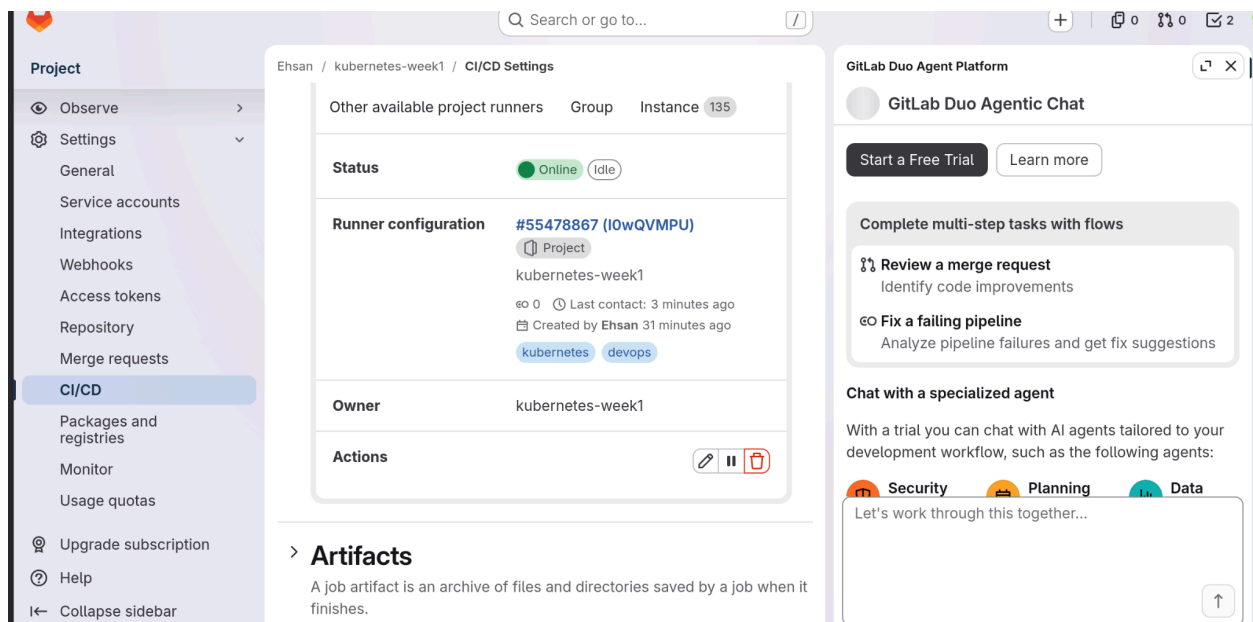
Identify code improvements

to fix a failing pipeline

Analyze pipeline failures and get fix suggestions

Chat with a specialized agent

With a tier you can chat with AI agents tailored to your development workflow such as the following agents:



بعد از نصب و در دسترس قرار گرفتن Image روی Worker Node ها، Pod مربوط به GitLab Runner اجرا شد و Runner در پنل GitLab به صورت Online نمایش داده شد. این وضعیت نشان داد که Runner Manager توانسته است با GitLab ارتباط برقرار کند و Token ذخیره شده در Secret معتبر است.

لیست تمامی پادها:

```
server04-01@server04:~/kubernetes-week1/manifests/gitlab-runner$ kubectl get pods -A -o wide
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	A
AGE	IP	NODE	NOMINATED NODE	READINESS GATES	
cattle-capi-system	capi-controller-manager-754dccb8b9-hspq5	1/1	Running	0	8
7m	10.244.0.7	server04	<none>	<none>	
cattle-fleet-local-system	fleet-agent-5cdb98c6fc-2p8wt	0/1	ImagePullBackOff	0	1
6m	10.244.2.68	server03	<none>	<none>	
cattle-fleet-local-system	fleet-agent-5cdb98c6fc-4m6qm	0/1	ContainerStatusUnknown	0	2
3m	10.244.2.62	server03	<none>	<none>	
cattle-fleet-local-system	fleet-agent-5cdb98c6fc-db6c7	0/1	ContainerStatusUnknown	0	2
1m	10.244.1.72	server02	<none>	<none>	
cattle-fleet-local-system	fleet-agent-5cdb98c6fc-kg8qp	0/1	ContainerStatusUnknown	0	7
3m	10.244.1.65	server02	<none>	<none>	
cattle-fleet-local-system	fleet-agent-5cdb98c6fc-l2h8d	0/1	Completed	0	8
2m	10.244.2.46	server03	<none>	<none>	
cattle-fleet-local-system	fleet-agent-5cdb98c6fc-nnvcb	0/1	ContainerStatusUnknown	1	9
0m	10.244.1.51	server02	<none>	<none>	
cattle-fleet-system	fleet-controller-785f8cff97-jzmrn	3/3	Running	0	9
4m	10.244.2.42	server03	<none>	<none>	
cattle-fleet-system	gitjob-6c65d4f7fd-wtf59	1/1	Running	0	9
4m	10.244.1.40	server02	<none>	<none>	
cattle-fleet-system	helmops-568b8774b8-ngz5k	1/1	Running	0	9
4m	10.244.1.39	server02	<none>	<none>	
cattle-system	rancher-76fd95cdcb-2l6qv	0/1	Error	0	1
10m	10.244.1.23	server02	<none>	<none>	
cattle-system	rancher-76fd95cdcb-4vxxz	1/1	Running	0	2
1m	10.244.1.68	server02	<none>	<none>	
cattle-system	rancher-76fd95cdcb-6vv29	0/1	Error	0	1
11m	10.244.2.28	server03	<none>	<none>	
cattle-system	rancher-76fd95cdcb-7p2qj	0/1	Evicted	0	7
3m	<none>	server02	<none>	<none>	

```
server04-01@server04:~/kubernetes-week1/manifests/gitlab-runner$ kubectl get services -A -o wide
```

NAMESPACE	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE	SELECTOR				
cattle-capi-system	capi-webhook-service	ClusterIP	10.100.4.107	<none>	443/TCP
89m	cluster.x-k8s.io/provider=cluster-api				
cattle-fleet-system	gitjob	ClusterIP	10.102.36.216	<none>	80/TCP
95m	app=gitjob				
cattle-fleet-system	monitoring-fleet-controller	ClusterIP	10.109.52.177	<none>	8080/TCP
95m	app=fleet-controller,fleet.cattle.io/shard-default=true				
cattle-fleet-system	monitoring-gitjob	ClusterIP	10.103.110.112	<none>	8081/TCP
95m	app=gitjob,fleet.cattle.io/shard-default=true				
cattle-system	imperative-api-extension	ClusterIP	10.97.171.166	<none>	6666/TCP
166m	app=rancher				
cattle-system	rancher	ClusterIP	10.105.207.161	<none>	80/TCP,443/TCP
3h11m	app=rancher				
cattle-system	rancher-internal	ClusterIP	10.109.99.74	<none>	443/TCP
3h11m	app=rancher				
cattle-system	rancher-webhook	ClusterIP	10.97.46.130	<none>	443/TCP
95m	app=rancher-webhook				
cert-manager	cert-manager	ClusterIP	10.103.12.152	<none>	9402/TCP
3h27m	app.kubernetes.io/component=controller,app.kubernetes.io/instance=cert-manager,app.kubernetes.io/name=cert-manager				
cert-manager	cert-manager-cainjector	ClusterIP	10.97.224.220	<none>	9402/TCP
3h27m	app.kubernetes.io/component=cainjector,app.kubernetes.io/instance=cert-manager,app.kubernetes.io/name=cainjector				
cert-manager	cert-manager-webhook	ClusterIP	10.109.99.0	<none>	443/TCP,9402/TCP
3h27m	app.kubernetes.io/component=webhook,app.kubernetes.io/instance=cert-manager,app.kubernetes.io/name=webhook				
default	kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
4h40m	<none>				
kube-system	kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP,53/TCP,9153/TCP
4h40m	k8s-app=kube-dns				
traefik	traefik	NodePort	10.107.84.62	192.168.122.204	80:30080/TCP,443:30443/TCP

```

server04-01@server04:~/kubernetes-week1/manifests/gitlab-runner$ helm list -A
NAME                                NAMESPACE              REVISION    UPDATED                               APP VERSION    STATUS    C
cert-manager                        cert-manager            1           2026-08-24 17:20:07.386049546 +0000 UTC deployed    c
ert-manager-v1.21.1                v1.21.1
fleet                               cattle-fleet-system     6           2026-08-24 19:32:03.572457944 +0000 UTC deployed    f
leet-110.0.0+up0.16.0             0.16.0
fleet-agent-local                  cattle-fleet-local-system 1           2026-08-24 19:17:26.260011696 +0000 UTC deployed    f
leet-agent-local-v0.0.0+s-8485a5dc16708bb7c6c863f3d8b5710a8fba51dfddb8712ac22b4c69b6fca
fleet-crd                          cattle-fleet-system     1           2026-08-24 19:11:32.544867244 +0000 UTC deployed    f
leet-crd-110.0.0+up0.16.0         0.16.0
gitlab-runner                      gitlab-runner           1           2026-08-24 20:03:35.303957218 +0000 UTC failed      g
itlab-runner-0.92.0               19.3.0
rancher                           cattle-system           4           2026-08-24 18:53:22.101645251 +0000 UTC deployed    r
ancher-2.15.0                     v2.15.0
rancher-turtles                   cattle-turtles-system   2           2026-08-24 19:14:12.139051779 +0000 UTC deployed    r
ancher-turtles-110.0.0+up0.27.0  0.27.0
server04-01@server04:~/kubernetes-week1/manifests/gitlab-runner$

```

جمع بندی

در این پروژه من Traefik را به عنوان Rancher، Ingress Controller را به عنوان ابزار مدیریت Kubernetes و GitLab Runner را به عنوان Runner متصل به GitLab نصب کردم. برای Traefik از Manifest های Kubernetes و برای Rancher و GitLab Runner از Helm Chart همراه با فایل Values استفاده کردم. Namespace، RBAC، Deployment، Service، IngressClass، Ingress Secret و Service ها هرکدام وظیفه مشخصی دارند و در کنار هم ساختار نهایی پروژه را تشکیل می دهند.

در پایان، Rancher از طریق HTTPS پاسخ **HTTP/2 200** داد، Pod اصلی آن در وضعیت **Running** قرار گرفت و GitLab Runner نیز در پنل GitLab به صورت **Online** نمایش داده شد.